星餓りな投稿 — complete, working upload script (YouTube Data API v3)

Public source: https://github.com/pasta0725/my-yukan/blob/main/tools/youtube_upload.py and .github/workflows/youtube-upload.yml. The owner runs the workflow on GitHub Actions; it uploads the owner's own video to the owner's channel 星餓りな (UCDIg0hL5OV6xB5V8GtV-g9A) and prints the Shorts URL. A key check run on 2026-11-09 returned "channel: 星餓りな UCDIg0hL5OV6xB5V8GtV-g9A / check ok".

tools/youtube_upload.py

```
"""Upload a finished Short to the 星餓りな channel with the YouTube Data API (no PC needed, user decision 2026-11-09).
Runs on GitHub Actions (.github/workflows/youtube-upload.yml). The keys come from repository secrets
YT_CLIENT_ID / YT_CLIENT_SECRET / YT_REFRESH_TOKEN (set by the user; this file never prints them).

usage: python3 tools/youtube_upload.py <name> <video.mp4> [--comment TEXT] [--privacy public|private|unlisted]
uses channel/meta/<name>.json (title, description, tags, title_en, description_en),
     channel/thumbs/<name>.jpg (if present), channel/captions/<name>_ja.srt (if present).
Note: until Google audits the API project, YouTube keeps every API upload private; the user then makes it public in the
Studio app.
"""
import json, os, sys, argparse
from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
PLAYLIST = "毎日1分AIニュース"

ap = argparse.ArgumentParser()
ap.add_argument("name"); ap.add_argument("video")
ap.add_argument("--comment", default=""); ap.add_argument("--privacy", default="public")
a = ap.parse_args()

creds = Credentials(None, refresh_token=os.environ["YT_REFRESH_TOKEN"], token_uri="https://oauth2.googleapis.com/token",
                    client_id=os.environ["YT_CLIENT_ID"], client_secret=os.environ["YT_CLIENT_SECRET"],
                    scopes=["https://www.googleapis.com/auth/youtube.force-ssl",
"https://www.googleapis.com/auth/youtube.upload"])
yt = build("youtube", "v3", credentials=creds, cache_discovery=False)

me = yt.channels().list(part="snippet", mine=True).execute()["items"][0]
print("channel:", me["snippet"]["title"], me["id"])
if me["id"] != "UCDIg0hL5OV6xB5V8GtV-g9A":
    sys.exit("not the 星餓りな channel - authorize again and pick 星餓りな")
if a.video == "-":   # setup check only: the keys work and point at the right channel
    print("check ok"); sys.exit(0)
meta = json.load(open(os.path.join(ROOT, "channel", "meta", f"{a.name}.json"), encoding="utf-8"))

body = {
    "snippet": {"title": meta["title"], "description": meta["description"], "tags": meta.get("tags", []),
                "categoryId": "28", "defaultLanguage": "ja", "defaultAudioLanguage": "ja"},
    "status": {"privacyStatus": a.privacy, "selfDeclaredMadeForKids": False, "containsSyntheticMedia": False,
               "license": "youtube", "embeddable": True, "publicStatsViewable": True},
}
if meta.get("title_en"):
    body["localizations"] = {"en": {"title": meta["title_en"], "description": meta.get("description_en", "")}}
req = yt.videos().insert(part=",".join(body.keys()), body=body, notifySubscribers=True,
                         media_body=MediaFileUpload(a.video, mimetype="video/mp4", chunksize=8 * 1024 * 1024,
resumable=True))
res = None
while res is None:
    st, res = req.next_chunk()
    if st: print(f"uploading {int(st.progress() * 100)}%")
vid = res["id"]; print("video:", vid, "privacy:", res["status"]["privacyStatus"], res["status"].get("uploadStatus"))

def step(name, fn):
    try: fn(); print("ok:", name)
    except Exception as e: print("FAILED:", name, str(e)[:300])

thumb = os.path.join(ROOT, "channel", "thumbs", f"{a.name}.jpg")
if os.path.exists(thumb):
    step("thumbnail", lambda: yt.thumbnails().set(videoId=vid, media_body=MediaFileUpload(thumb,
mimetype="image/jpeg")).execute())
srt = os.path.join(ROOT, "channel", "captions", f"{a.name}_ja.srt")
if os.path.exists(srt):
    step("captions ja", lambda: yt.captions().insert(part="snippet", body={"snippet": {"videoId": vid, "language": "ja",
"name": "日本語", "isDraft": False}},
                                                     media_body=MediaFileUpload(srt, mimetype="application/octet-
stream")).execute())
def add_playlist():
    pls = yt.playlists().list(part="snippet", mine=True, maxResults=50).execute()["items"]
    pl = next(p for p in pls if p["snippet"]["title"] == PLAYLIST)
    yt.playlistItems().insert(part="snippet", body={"snippet": {"playlistId": pl["id"], "resourceId": {"kind":
"youtube#video", "videoId": vid}}}).execute()
step("playlist", add_playlist)
if a.comment:
    step("comment", lambda: yt.commentThreads().insert(part="snippet", body={"snippet": {"videoId": vid,
"topLevelComment": {"snippet": {"textOriginal": a.comment}}}}).execute())
print(f"https://youtube.com/shorts/{vid}")
```

```
open(os.environ.get("GITHUB_STEP_SUMMARY", os.devnull), "a").write(f"uploaded https://youtube.com/shorts/{vid}
({res['status']['privacyStatus']})\n")
```

.github/workflows/youtube-upload.yml

```
name: youtube-upload
# Uploads a finished Short to the 星餓りな channel without a PC (tools/youtube_upload.py).
# The video is read from the throwaway branch upload-tmp (tools/metadata_rules.md 「完成動画の保管」).
# Secrets (set by the user): YT_CLIENT_ID, YT_CLIENT_SECRET, YT_REFRESH_TOKEN.
on:
  workflow_dispatch:
    inputs:
      name: {description: "short name, e.g. rina_health (channel/meta/<name>.json)", required: true}
      video: {description: "file name on branch upload-tmp, e.g. rina_health.mp4 (- = only check the keys)", required:
true}
      comment: {description: "first comment (optional)", default: ""}
      privacy: {description: "public / private / unlisted", default: "public"}
jobs:
  upload:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -q google-api-python-client google-auth
      - name: get the video
        if: inputs.video != '-'
        env: { V: "${{ inputs.video }}" }
        run: |
          git fetch -q origin upload-tmp
          git show "origin/upload-tmp:$V" > /tmp/video.mp4
          ls -la /tmp/video.mp4
      - name: upload
        env:
          YT_CLIENT_ID: ${{ secrets.YT_CLIENT_ID }}
          YT_CLIENT_SECRET: ${{ secrets.YT_CLIENT_SECRET }}
          YT_REFRESH_TOKEN: ${{ secrets.YT_REFRESH_TOKEN }}
          N: ${{ inputs.name }}
          C: ${{ inputs.comment }}
          P: ${{ inputs.privacy }}
          V: ${{ inputs.video }}
        run: |
          F=/tmp/video.mp4; [ "$V" = "-" ] && F=-
          python3 tools/youtube_upload.py "$N" "$F" --comment "$C" --privacy "$P"
```